VOLSCOPE — RALPH LOOP PROMPT

This is a Ralph Wiggum prompt. Each iteration, you get a fresh context.

You orient yourself by reading the filesystem. You work. You exit.

The loop re-feeds this prompt. You continue where you left off.

COMPLETION SIGNAL: Output <promise>VOLSCOPE_COMPLETE</promise>

ONLY when ALL tasks in progress.json are "done" AND all tests pass.

BLOCKED SIGNAL: Output <promise>BLOCKED</promise> with explanation

if you hit something that requires human judgment.

STEP 0: ORIENT YOURSELF (do this EVERY iteration)

```bash
# 1. Read progress
cat progress.json 2>/dev/null || echo "FIRST_RUN"

# 2. Check what exists
ls -la volscope/ 2>/dev/null
ls -la tests/ 2>/dev/null

# 3. Check test status
cd volscope && python -m pytest tests/ -v --tb=short 2>/dev/null; cd ..

# 4. Check if app runs
timeout 10 streamlit run volscope/ui/app.py --server.headless true 2>&1 | head -5
```

If progress.json doesn't exist → you're on FIRST_RUN → start Phase 1.
If it exists → read it, find the first task with status != "done", work on that.
If ALL tasks are "done" AND pytest passes AND app starts → output the completion promise.

STEP 1: FIRST RUN SETUP

On first run, create this exact project structure and progress.json:

```bash
mkdir -p volscope/{data,analytics,ui/{pages,components,styles},utils} tests scripts
touch volscope/__init__.py volscope/data/__init__.py volscope/analytics/__init__.py
touch volscope/ui/__init__.py volscope/ui/pages/__init__.py volscope/ui/components/_
touch volscope/utils/__init__.py tests/__init__.py
```

Create `progress.json`:

```json
```

```
{
  "project": "VolScope",
  "created": "<timestamp>",
  "phases": {
    "P1_foundation": {
      "tasks": {
        "T01_requirements": "pending",
        "T02_config": "pending",
        "T03_black_scholes": "pending",
        "T04_black_scholes_tests": "pending",
        "T05_historical_vol": "pending",
        "T06_historical_vol_tests": "pending",
        "T07_vol_metrics": "pending",
        "T08_vol_metrics_tests": "pending"
      }
    },
    "P2_data": {
      "tasks": {
        "T09_database_schema": "pending",
        "T10_ticker_universe": "pending",
        "T11_price_fetcher": "pending",
        "T12_price_fetcher_tests": "pending",
        "T13_options_scraper": "pending",
        "T14_options_scraper_tests": "pending",
        "T15_seed_script": "pending",
        "T16_data_validation": "pending"
      }
    },
    "P3_analytics": {
      "tasks": {
        "T17_spread_analysis": "pending",
        "T18_opportunity_engine": "pending",
        "T19_crowded_trades": "pending",
        "T20_analytics_tests": "pending"
      }
    },
    "P4_ui": {
      "tasks": {
        "T21_theme_css": "pending",
        "T22_chart_builders": "pending",
        "T23_metric_components": "pending",
        "T24_scope_page": "pending",
        "T25_scan_page": "pending",
        "T26_discover_page": "pending",
        "T27_app_main": "pending",
        "T28_sidebar": "pending"
      }
    },
    "P5_integration": {
      "tasks": {
        "T29_integration_tests": "pending",
        "T30_error_handling": "pending",
        "T31_caching": "pending",
        "T32_makefile": "pending",
        "T33_dockerfile": "pending",
        "T34_readme": "pending",
        "T35_final_validation": "pending"
      }
    }
  }
}
```

Create `CLAUDE.md`:

```
markdown
```

```
# VolScope — Developer Guide

## What is this?
Volatility Intelligence Platform for retail options traders.
Python-only. Streamlit frontend. DuckDB backend.

## Commands
- `make setup` — Install dependencies
- `make test` — Run all tests
- `make run` — Start Streamlit app
- `make scrape` — Run daily data scrape
- `pytest tests/ -v` — Verbose test output
- `streamlit run volscope/ui/app.py` — Start app

## Architecture
- `volscope/analytics/` — All financial math (BSM, HV, metrics)
- `volscope/data/` — Data pipeline (scraper, DB, fetcher)
- `volscope/ui/` — Streamlit pages and components
- `tests/` — pytest test suite

## Conventions
- Type hints everywhere
- Docstrings on every public function
- Tests before implementation (TDD)
- All financial calculations validated against known values
- Edge cases handled gracefully (return None, not crash)
```

Then mark T01 as in_progress and continue.

TASK SPECIFICATIONS

T01: requirements.txt

Create with PINNED versions:

```
numpy>=1.24.0,<2.0.0
pandas>=2.0.0,<3.0.0
scipy>=1.10.0,<2.0.0
yfinance>=0.2.31
duckdb>=0.9.0,<2.0.0
streamlit>=1.28.0,<2.0.0
plotly>=5.17.0,<6.0.0
pytest>=7.4.0
pytest-cov>=4.1.0
rich>=13.0.0
python-dotenv>=1.0.0
requests>=2.31.0
```

Run `pip install -r requirements.txt`. If any fail, fix the version. **DONE WHEN:** `pip install -r requirements.txt` exits 0.

T02: config.py

```
python
```

```
"""Central configuration. All magic numbers live here."""
from pathlib import Path

# Paths
PROJECT_ROOT = Path(__file__).parent.parent
DATA_DIR = PROJECT_ROOT / "data"
DB_PATH = DATA_DIR / "volscope.db"

# Annualization
TRADING_DAYS_PER_YEAR = 252
CALENDAR_DAYS_PER_YEAR = 365

# Default Windows
DEFAULT_HV_SHORT = 20
DEFAULT_HV_LONG = 60
DEFAULT_RANK_LOOKBACK = 252  # 52 weeks

# Scraper
SCRAPE_DELAY_SECONDS = 1.5  # between tickers, respect Yahoo
MAX_RETRIES = 3
REQUEST_TIMEOUT = 10

# UI
APP_TITLE = "◈ VolScope"
DEFAULT_TICKER = "SPY"
```

DONE WHEN: `python -c "from volscope.config import *; print(DB_PATH)"` works.

T03: black_scholes.py (analytics/)

Implement the COMPLETE Black-Scholes-Merton suite:

```python
"""
Black-Scholes-Merton Option Pricing & IV Solver.

All functions use continuous dividend yield q.
If no dividend, pass q=0.

Functions:
    bs_price(S, K, T, r, sigma, q=0, option_type='call') -> float
    bs_vega(S, K, T, r, sigma, q=0) -> float
    bs_delta(S, K, T, r, sigma, q=0, option_type='call') -> float
    bs_gamma(S, K, T, r, sigma, q=0) -> float
    bs_theta(S, K, T, r, sigma, q=0, option_type='call') -> float
    implied_volatility(market_price, S, K, T, r, q=0, option_type='call') -> float|N
"""
import numpy as np
from scipy.stats import norm
```

CRITICAL IMPLEMENTATION DETAILS:

1. **d1 and d2:**

```
d1 = (ln(S/K) + (r - q + σ²/2) × T) / (σ × √T)
d2 = d1 - σ × √T
```

2. **IV Solver — Newton-Raphson with fallback to Bisection:**
   - Start with initial guess $\sigma_0 = 0.3$
   - Newton: σ_{n+1} = σ_n - (BSM(σ_n) - P_market) / Vega(σ_n)
   - If Vega < 1e-12: switch to Bisection on [0.001, 5.0]
   - Max 100 iterations, tolerance 1e-8
   - Return None if: T ≤ 0, price ≤ 0, price < intrinsic, no convergence

3. **Edge cases to handle:**
    - T = 0 → return intrinsic value for price, None for IV
    - S = 0 or K = 0 → return 0 / None
    - Very deep ITM/OTM → Bisection fallback
    - Negative price input → return None

DONE WHEN: T04 tests all pass.

T04: black_scholes TESTS

```python
```

```python
"""Tests for BSM model — validated against known analytical solutions."""
import pytest
import numpy as np
from volscope.analytics.black_scholes import (
    bs_price, bs_vega, bs_delta, bs_gamma, bs_theta, implied_volatility
)

class TestBSMPricing:
    """Test option pricing against known values."""

    def test_atm_call(self):
        # S=100, K=100, T=1, r=0.05, σ=0.20, q=0
        price = bs_price(100, 100, 1.0, 0.05, 0.20, option_type='call')
        assert abs(price - 10.4506) < 0.01

    def test_atm_put(self):
        price = bs_price(100, 100, 1.0, 0.05, 0.20, option_type='put')
        assert abs(price - 5.5735) < 0.01

    def test_put_call_parity(self):
        S, K, T, r, sigma = 100, 100, 1.0, 0.05, 0.30
        call = bs_price(S, K, T, r, sigma, option_type='call')
        put = bs_price(S, K, T, r, sigma, option_type='put')
        # C - P = S*exp(-qT) - K*exp(-rT)
        assert abs((call - put) - (S - K * np.exp(-r * T))) < 0.01

    def test_deep_itm_call(self):
        price = bs_price(150, 100, 1.0, 0.05, 0.20, option_type='call')
        assert price > 50  # at least intrinsic

    def test_deep_otm_call(self):
        price = bs_price(50, 100, 1.0, 0.05, 0.20, option_type='call')
        assert price < 1  # nearly worthless

    def test_zero_time(self):
        call = bs_price(105, 100, 0.0, 0.05, 0.20, option_type='call')
        assert abs(call - 5.0) < 0.01  # intrinsic

class TestGreeks:
    def test_vega_positive(self):
        v = bs_vega(100, 100, 1.0, 0.05, 0.20)
        assert v > 0
        assert abs(v - 39.45) < 0.5

    def test_atm_delta_call(self):
        d = bs_delta(100, 100, 1.0, 0.05, 0.20, option_type='call')
        assert 0.5 < d < 0.7  # ATM call delta slightly > 0.5

    def test_atm_delta_put(self):
        d = bs_delta(100, 100, 1.0, 0.05, 0.20, option_type='put')
        assert -0.5 > d > -0.7

class TestIVSolver:
    def test_iv_recovery_atm(self):
        """Generate price with known vol, recover it."""
        true_vol = 0.25
        price = bs_price(100, 100, 0.5, 0.05, true_vol, option_type='call')
        recovered = implied_volatility(price, 100, 100, 0.5, 0.05, option_type='call
        assert recovered is not None
        assert abs(recovered - true_vol) < 1e-6

    def test_iv_recovery_otm(self):
        true_vol = 0.40
        price = bs_price(100, 120, 0.25, 0.05, true_vol, option_type='call')
        recovered = implied_volatility(price, 100, 120, 0.25, 0.05, option_type='cal
        assert recovered is not None
        assert abs(recovered - true_vol) < 1e-4
```

```
    def test_iv_recovery_put(self):
        true_vol = 0.30
        price = bs_price(100, 100, 1.0, 0.05, true_vol, option_type='put')
        recovered = implied_volatility(price, 100, 100, 1.0, 0.05, option_type='put'
        assert recovered is not None
        assert abs(recovered - true_vol) < 1e-6

    def test_iv_near_zero_price(self):
        result = implied_volatility(0.001, 100, 200, 0.1, 0.05, option_type='call')
        # Should return something or None, but NOT crash
        assert result is None or result > 0

    def test_iv_negative_price(self):
        result = implied_volatility(-5, 100, 100, 1.0, 0.05)
        assert result is None

    def test_iv_zero_time(self):
        result = implied_volatility(5, 105, 100, 0.0, 0.05)
        assert result is None

    def test_iv_below_intrinsic(self):
        # Call price below intrinsic should return None
        result = implied_volatility(3.0, 105, 100, 1.0, 0.05, option_type='call')
        assert result is None

    @pytest.mark.parametrize("vol", [0.05, 0.10, 0.20, 0.50, 1.00, 2.00])
    def test_iv_range_of_vols(self, vol):
        """IV solver must work across wide range of volatilities."""
        price = bs_price(100, 100, 0.5, 0.05, vol, option_type='call')
        recovered = implied_volatility(price, 100, 100, 0.5, 0.05, option_type='call
        assert recovered is not None
        assert abs(recovered - vol) < 1e-4
```

Run: `pytest tests/test_black_scholes.py -v` **DONE WHEN:** ALL tests pass. Zero failures. Zero errors.

T05: historical_vol.py (analytics/)

Four estimators. All annualized. All return percentage (e.g., 25.0 for 25%).

```
python

"""
Historical Volatility Estimators.

All functions take a pandas Series of OHLCV data and return annualized vol in %.

Estimators:
    hv_close_to_close(close: pd.Series, window: int, ann: int = 252) -> pd.Series
    hv_parkinson(high: pd.Series, low: pd.Series, window: int, ann: int = 252) -> pd
    hv_garman_klass(open, high, low, close: pd.Series, window: int, ann: int = 252)
    hv_yang_zhang(open, high, low, close: pd.Series, window: int, ann: int = 252) ->
"""
```

Key implementation notes:

- Yang-Zhang k = 0.34 / (1.34 + (n+1)/(n-1))
- All estimators return pd.Series (rolling window), NOT a single float
- Handle NaN gracefully (first `window` values will be NaN)
- Multiply by 100 to return percentage
- Garman-Klass: $\sigma^2 = (1/n) \times \Sigma[0.5 \times \ln(H/L)^2 - (2\ln 2 - 1) \times \ln(C/O)^2] \times ann$

T06: historical_vol TESTS

```python
"""Generate synthetic GBM paths with KNOWN volatility, recover it."""
import numpy as np
import pandas as pd

def generate_gbm(S0=100, mu=0.05, sigma=0.25, dt=1/252, N=504):
    """Generate 2 years of synthetic GBM data with known sigma."""
    np.random.seed(42)  # reproducible
    Z = np.random.standard_normal(N)
    log_returns = (mu - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * Z
    prices = S0 * np.exp(np.cumsum(log_returns))
    prices = np.insert(prices, 0, S0)

    # Generate OHLC from close prices (simplified)
    close = pd.Series(prices)
    open_ = close.shift(1).fillna(S0)
    high = pd.DataFrame({'c': close, 'o': open_}).max(axis=1) * (1 + np.abs(np.randc
    low = pd.DataFrame({'c': close, 'o': open_}).min(axis=1) * (1 - np.abs(np.random

    return open_, high, low, close

class TestHVEstimators:
    def setup_method(self):
        self.open, self.high, self.low, self.close = generate_gbm(sigma=0.25)

    def test_cc_recovers_vol(self):
        hv = hv_close_to_close(self.close, window=60)
        recent = hv.dropna().tail(100).mean()
        assert 15 < recent < 40  # 25% ± tolerance for stochastic

    def test_yz_lowest_error(self):
        """Yang-Zhang should have lowest MSE vs true vol."""
        cc = hv_close_to_close(self.close, 60).dropna().tail(100)
        yz = hv_yang_zhang(self.open, self.high, self.low, self.close, 60).dropna().
        cc_err = (cc - 25.0).abs().mean()
        yz_err = (yz - 25.0).abs().mean()
        assert yz_err <= cc_err * 1.5  # YZ should be at least competitive

    def test_all_return_series(self):
        for fn in [hv_close_to_close, hv_parkinson, hv_garman_klass, hv_yang_zhang]:
            # ... assert isinstance result is pd.Series
            pass

    def test_nan_handling(self):
        hv = hv_close_to_close(self.close, window=20)
        assert hv.iloc[:19].isna().all()  # first window-1 values are NaN
        assert hv.iloc[20:].notna().all()  # rest are valid
```

T07: vol_metrics.py (analytics/)

```python
"""
IV Rank, IV Percentile, Vol Regime, IV-HV Spread Analysis.

Functions:
    iv_rank(current: float, history: array-like) -> float  # 0-100
    iv_percentile(current: float, history: array-like) -> float  # 0-100
    vol_regime(current: float, history: array-like, z_thresh: float = 1.5) -> dict
    iv_hv_spread(iv: float, hv: float, spread_history: array-like = None) -> dict
"""
```

iv_rank: `(current - min) / (max - min) * 100`. Return 50.0 if max == min. iv_percentile: `count(history < current) / len(history) * 100` vol_regime: Returns `{"regime": "HIGH"|"LOW"|"NORMAL", "z_score": float, "mean": float, "std": float}` iv_hv_spread: Returns `{"spread": float, "ratio": float, "signal": "RICH"|"CHEAP"|"NEUTRAL", "z_score": float|None}`

- RICH if spread > 0 and ratio > 1.1
- CHEAP if spread < 0 and ratio < 0.9
- NEUTRAL otherwise

T08: vol_metrics TESTS

Test edge cases: empty history, single value history, all same values, NaN in history.
Test that iv_rank always returns 0-100.
Test that iv_percentile of a value higher than all history = ~100.

T09-T16: DATA PIPELINE

T09: database.py — Use DuckDB. Create tables on init if they don't exist.

```python
class VolScopeDB:
    def __init__(self, db_path: str = None):
        self.db_path = db_path or str(DB_PATH)
        self.con = duckdb.connect(self.db_path)
        self._create_tables()

    def _create_tables(self):
        """Create all tables if they don't exist."""
        # daily_vol, earnings, options_snapshots (schemas from previous spec)

    def upsert_daily(self, ticker, date, **kwargs):
        """Insert or update daily vol record."""

    def get_ticker_history(self, ticker, start=None, end=None) -> pd.DataFrame:
        """Get full history for a ticker."""

    def get_all_latest(self) -> pd.DataFrame:
        """Get most recent record for ALL tickers. Used by Scanner."""

    def get_available_tickers(self) -> list[str]:
        """List all tickers with data."""
```

T10: ticker_universe.py — Hardcoded list of ~80 liquid tickers (from prev spec).

T11: price_fetcher.py — Fetch OHLCV via yfinance.

```python
def fetch_ohlcv(ticker: str, period: str = "2y") -> pd.DataFrame:
    """Fetch OHLCV data. Returns DataFrame with columns: Open, High, Low, Close, Vol
    # Use yfinance. Handle errors: ticker not found, no data, API timeout.
    # Return empty DataFrame on error, not crash.
```

T12: price_fetcher tests — Test with real ticker (SPY), verify columns exist, no crash on invalid ticker.

T13: options_scraper.py — THE CRITICAL MODULE.

```python
```

```python
def scrape_options_chain(ticker: str) -> dict | None:
    """
    Scrape Yahoo options chain, compute ATM IV.

    Process:
    1. Get all expiry dates
    2. Find the two expiries closest to 30 days
    3. For each: get options chain, find ATM strikes
    4. Compute IV from bid/ask midpoint using BSM solver
    5. Interpolate temporally to get 30-day ATM IV
    6. Also compute: total call/put volume, OI, P/C ratio

    Returns dict:
    {
        'iv_30d': float,
        'iv_60d': float | None,
        'total_call_volume': int,
        'total_put_volume': int,
        'put_call_ratio': float,
        'total_open_interest': int,
        'spot_price': float,
    }

    Returns None on failure (log the error, don't crash).

    YAHOO GOTCHAS:
    - yfinance.Ticker(sym).options → tuple of expiry date strings
    - yfinance.Ticker(sym).option_chain(expiry) → namedtuple(calls, puts)
    - Each is a DataFrame with: strike, bid, ask, lastPrice, volume,
      openInterest, impliedVolatility
    - DO NOT use Yahoo's impliedVolatility column — compute yourself
    - Filter: bid > 0 AND ask > 0 AND (volume > 0 OR openInterest > 10)
    - ATM = strikes closest to spot price
    - For IV: use mid = (bid + ask) / 2 as market price input to BSM solver
    - T = calendar days to expiry / 365
    - r = risk-free rate (hardcode 0.045 for now, or fetch ^IRX)
    """
```

T14: scraper tests — Test with SPY (always has options). Verify iv_30d is between 5% and 80%. Verify no crash on ticker without options.

T15: seed_script.py — Script that backfills HV data for all tickers (2 years of OHLCV is free from Yahoo). Computes and stores HV metrics.

T16: data_validation.py — Check for: NaN IVs, IV < 1% or > 500%, missing dates, duplicate entries.

T17-T20: ANALYTICS ENGINE

T17: spread_analysis.py

python

```python
def compute_iv_hv_timeseries(iv_series, hv_series) -> pd.DataFrame:
    """Compute spread, ratio, z-score over time."""

def detect_spread_extremes(spread_series, threshold_z=2.0) -> pd.DataFrame:
    """Find dates where spread was extreme."""
```

T18: opportunity.py — THE "PAYPAL MOMENT" ENGINE

python

```python
def find_cheapest_vol(latest_data: pd.DataFrame, n: int = 5) -> pd.DataFrame:
    """Top N tickers with lowest IV Percentile.
    Adds context column explaining WHY it's cheap."""

def find_richest_premium(latest_data: pd.DataFrame, n: int = 5) -> pd.DataFrame:
    """Top N tickers with highest IV Percentile.
    Flags if earnings are upcoming (expected high IV)."""

def find_daily_outliers(latest_data: pd.DataFrame, prev_data: pd.DataFrame, n: int =
    """Top N tickers with largest 1-day IV change, normalized by typical daily chang
```

T19: crowded_trades.py

```python
def compute_crowded_score(row: pd.Series, historical: pd.DataFrame) -> float:
    """
    Composite crowding score 0-100 based on:
    - Put/Call ratio z-score (25%)
    - Open Interest vs 20-day avg (25%)
    - Volume vs 20-day avg (25%)
    - IV-HV spread z-score (25%)
    """
```

T20: analytics tests — Test all analytics functions. Edge cases: empty data, single row, all zeros.

T21-T28: STREAMLIT UI

T21: theme.py — Dark terminal theme CSS (from previous spec). Inject via st.markdown().

T22: chart_builders.py — Plotly charts. CRITICAL SPECS:

```python
```

```python
def create_iv_hv_chart(history: pd.DataFrame, ticker: str) -> go.Figure:
    """
    Main IV vs HV chart.

    - IV line: solid, color #00d4aa, width 2.5
    - HV line: dashed, color #5b8cff, width 1.5
    - Fill between: red when IV > HV, green when HV > IV (subtle, opacity 0.1)
    - Earnings markers: vertical dashed lines, gold (#ffd700), opacity 0.5
    - Background: #0a0b0f
    - Grid: #1e2030
    - Range selector buttons: 1M, 3M, 6M, 1Y, ALL
    - Hover mode: x unified
    - Template: plotly_dark (customized)

    USE plotly.graph_objects, NOT plotly.express.
    """

def create_spread_chart(history: pd.DataFrame) -> go.Figure:
    """
    IV-HV Spread bar chart.

    - Positive bars: #ff4466 (options rich)
    - Negative bars: #00d4aa (options cheap)
    - Reference line at 0
    - Same x-axis range as IV chart
    """

def create_percentile_chart(history: pd.DataFrame) -> go.Figure:
    """
    IV Percentile area chart (0-100%).

    - Area fill with gradient
    - Horizontal reference lines at 20% and 80%
    - Background color zones: green below 20, red above 80
    """
```

T23: metric_components.py — Streamlit metric cards and KPI displays.

T24: scope_page.py — The IV Chart page.

```python
def render_scope_page(db: VolScopeDB, ticker: str, settings: dict):
    """
    Layout:
    Row 1: [Current IV] [HV 20d] [IV Rank] [IV Perc] [IV-HV Spread]
    Row 2: 52-week IV range bar
    Row 3: IV vs HV chart (full width)
    Row 4: IV-HV Spread chart (full width)
    Row 5: IV Percentile chart (full width)
    """
```

T25: scan_page.py — The Scanner.

```python
```

```
def render_scan_page(db: VolScopeDB, settings: dict):
    """
    Sortable dataframe with columns:
    Ticker | Sector | IV | IV Rank | IV Perc | HV 20d | Spread | P/C Ratio | 1d Δ

    Color-coded: red background for IV Perc > 80, green for < 20.
    Click on row → updates session state → navigates to Scope page.

    Filters in expander:
    - Min/Max IV Percentile
    - Sector multiselect
    - Spread direction (Rich/Cheap/All)
    """
```

T26: discover_page.py — The Opportunity Dashboard.

```python
def render_discover_page(db: VolScopeDB, settings: dict):
    """
    Four sections in 2x2 grid:

    Top-left: 💎 CHEAPEST VOL (5 cards)
    Top-right: 🔥 RICHEST PREMIUM (5 cards)
    Bottom-left: ⚡ TODAY'S VOL MOVERS (5 cards)
    Bottom-right: 🎯 CROWDED TRADES (top 10 with heatmap bars)

    Each card is clickable → navigates to Scope for that ticker.

    Cards show:
    - Ticker + Name
    - IV Percentile pill (color-coded)
    - One-line context: "IV at 12.3% — cheaper than 97% of past year"
    - IV-HV Spread indicator
    """
```

T27: app.py — Main entry point.

```python
"""
streamlit run volscope/ui/app.py

- st.set_page_config(page_title="VolScope", layout="wide", initial_sidebar_state="ex
- Inject custom CSS from theme
- Initialize DB connection (cached)
- Sidebar: ticker search, page navigation, settings
- Main area: render selected page
- Session state: selected_ticker, active_page, settings dict
"""
```

T28: sidebar.py — Sidebar with ticker autocomplete, navigation, HV settings.

T29-T35: INTEGRATION & POLISH

T29: Integration tests — End-to-end: fetch data for SPY, compute all metrics, verify charts render without error.

T30: Error handling audit — Every function must handle: None input, empty DataFrame, API timeout, missing columns. NO UNCAUGHT EXCEPTIONS.

T31: Caching — st.cache_data on all DB queries (TTL=3600). st.cache_resource on DB connection.

T32: Makefile

```makefile
```

```
.PHONY: setup test run scrape clean

setup:
    pip install -r requirements.txt

test:
    python -m pytest tests/ -v --tb=short

run:
    streamlit run volscope/ui/app.py

scrape:
    python scripts/daily_scrape.py

seed:
    python scripts/seed_database.py

clean:
    rm -f data/volscope.db
```

T33: Dockerfile

dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN python scripts/seed_database.py --tickers SPY,AAPL,TSLA,NVDA,META
EXPOSE 8501
CMD ["streamlit", "run", "volscope/ui/app.py", "--server.port=8501", "--server.addre
```

T34: README.md — Must include:

- One-line description
- Hero screenshot (placeholder path)
- Feature list
- Quick start (3 commands: clone, install, run)
- Architecture diagram (ASCII)
- How to add tickers
- How to set up daily scraping (cron)
- License (MIT)

T35: Final Validation — Run this EXACT sequence:

bash

```
# 1. Clean install
rm -rf data/volscope.db
pip install -r requirements.txt

# 2. All tests pass
python -m pytest tests/ -v --tb=short
# MUST BE: all passed, 0 failed, 0 errors

# 3. Seed database
python scripts/seed_database.py --tickers SPY,AAPL,TSLA
# MUST BE: exits 0, data/volscope.db exists and has data

# 4. App starts
timeout 15 streamlit run volscope/ui/app.py --server.headless true 2>&1 | grep -q "Y
# MUST BE: "You can now view your Streamlit app"

# 5. Scraper works
python scripts/daily_scrape.py --tickers SPY
# MUST BE: exits 0, new data in DB
```

If ALL five checks pass → output: <promise>VOLSCOPE_COMPLETE</promise> If any fail → fix it, re-run, loop.

RULES FOR EVERY ITERATION

1. Read progress.json FIRST. Find the first non-"done" task. Work on it.
2. After completing a task: run its tests. If green → update progress.json to "done".
3. If tests fail: fix the code, re-run. Do NOT mark as done until tests pass.
4. If blocked (e.g., Yahoo API down, unclear requirement): output <promise>BLOCKED</promise>.
5. NEVER skip a task. NEVER mark as done without verification.
6. NEVER use Yahoo's impliedVolatility column. Compute IV yourself.
7. Commit after each completed task: `git add -A && git commit -m "T##: description"`
8. Update CLAUDE.md if you make architectural decisions.

COMPLETION PROMISE

When ALL tasks in progress.json are "done" AND the final validation (T35) passes:

<promise>VOLSCOPE_COMPLETE</promise>